

Frontend Architecture & System Design with React

State Modeling • API Contracts • Scalability • Security

MiniTasks Project — Academic Deep Dive

Lecture 6

Learning Objectives

By the End of This Lecture You Will Be Able To:

- ▶ Model a frontend system before coding
- ▶ Separate responsibilities correctly
- ▶ Design scalable React architecture
- ▶ Understand state formally
- ▶ Build an API abstraction layer
- ▶ Analyze architectural trade-offs



Why Architecture Before Code?

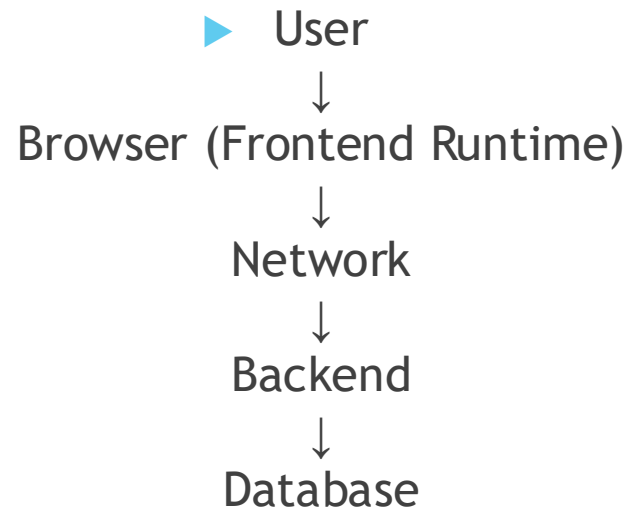
Why We Do NOT Start With React

- ▶ Code without structure leads to chaos
- ▶ UI reflects state
- ▶ State reflects domain
- ▶ Architecture determines scalability
- ▶ Formula:
- ▶ $UI = f(state)$

```
mirror_mod = modifier_ob.  
# Set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True  
  
# selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
  
print("please select exactly  
  
-- OPERATOR CLASSES --  
  
types.Operator):  
on X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
  
context):  
context.active_object is not
```

System Perspective

▶ Frontend as a Distributed System Client



What This Means for Us

- Each request is **stateless** (token required every time)
- Network has **latency** (we need loading & error states)
- Frontend and backend must follow an **API contract**
- Security is enforced via **CORS + JWT**

Lecture Structure



System Modeling



Architectural Principles



Data Flow Formalization



Step-by-Step Implementation



Error Handling & Debugging



Scalability & Trade-offs



Rule of the lecture:



Think → Structure → Implement → Validate

BLOCK 1 — THINKING BEFORE CODING

We Do NOT Start With Code

Before opening VS Code, we must understand:

- ▶ What are we building?
- ▶ Who is the user?
- ▶ What problem are we solving?
- ▶ What data exists?
- ▶ What states exist?

Programming is applied system thinking.

What Are We Building?

MiniTasks system.

It includes:

- ▶ User authentication
- ▶ Task management
- ▶ Filtering
- ▶ Protected access
- ▶ API communication

This is not “a React app”.

- ▶ It is a **task management system with authentication.**

Responsibility Separation

We divide the system into two layers:

Frontend Responsibilities

- ▶ Render UI
- ▶ Manage UI state
- ▶ Send API requests
- ▶ Handle loading & errors

Backend Responsibilities (later)

- ▶ Store tasks
- ▶ Validate token
- ▶ Manage users
- ▶ Enforce permissions

This prevents confusion later.

Define Data Model First

Before React, define the model:

Task

- ▶ id: string
- ▶ title: string
- ▶ done: boolean

Auth

- ▶ token: string
- ▶ If we don't define this first, UI becomes chaotic.

User Scenarios

What does the user do?

- ▶ Register
- ▶ Login
- ▶ See tasks
- ▶ Create task
- ▶ Toggle task
- ▶ Delete task
- ▶ Filter tasks

These scenarios define our UI.

Application States

What states must the system handle?

- ▶ Not authenticated
- ▶ Authenticated
- ▶ Loading
- ▶ Error
- ▶ Empty list
- ▶ Task updating

React = state management engine.

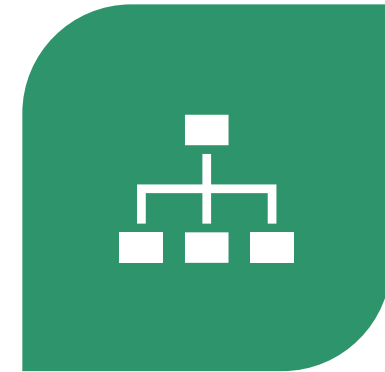
Checkpoint — What we have so far (No code yet)



WE UNDERSTAND: DATA
MODEL, SCENARIOS, STATES



NEXT: WE WILL IMPLEMENT
IN SMALL STEPS



RULE: “IF YOU ARE LOST,
RETURN TO THE
STRUCTURE SLIDE.”

Decomposition Into Parts

Now we divide into screens:

Pages

- ▶ Login
- ▶ Register
- ▶ Tasks

Components

- ▶ Navbar
- ▶ TaskItem
- ▶ TaskList
- ▶ ProtectedRoute

Only now we are ready for architecture.

Contract of the lesson

We code in order:

router → authStore → ProtectedRoute →
API client → pages → tasks

We never jump between files randomly

BLOCK 2 — FRONTEND ARCHITECTURE

Project Structure (Before Writing Code)

```
src/  
  api/  
  components/  
  pages/  
  store/  
  styles/  
  App.jsx  
  index.js
```

Why This Structure?

- pages → route-level screens
- components → reusable UI
- api → server communication
- store → auth logic
- styles → CSS

Rule:

UI must not contain fetch logic.

Why structure first?

- ▶ avoids “spaghetti imports”
- ▶ each folder has responsibility

Common mistake

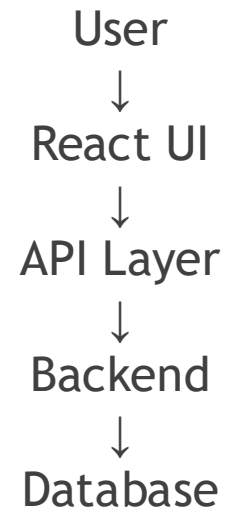
- ▶ Students create files in random places
Fix: show correct path + naming rule

Checkpoint

- ▶ Everyone has the folders created.

Data Flow Architecture

System flow:



React never talks directly to database.

Why We Need API Layer

Without API layer:

- ▶ fetch scattered everywhere
- ▶ token logic duplicated
- ▶ error handling inconsistent

With API layer:

- ▶ single source of truth
- ▶ centralized token injection
- ▶ clean components

Why ProtectedRoute?

We separate:

- ▶ UI rendering

vs

Authorization logic

- ▶ This keeps responsibilities clean.

BLOCK 3 STEP-BY-STEP IMPLEMENTATION

We write files in the same order as the architecture.

Create Project (Choose your setup: CRA or other)

- ▶ `npx create-react-app minitasks`
`cd minitasks`
`npm start`

Install router:

- ▶ `npm install react-router-dom`

Setup (CRA, not Vite)

Tools & Setup (CRA)

- Node.js, npm
- create-react-app
- react-router-dom

Checkpoint

React default page opens at localhost:3000

Common Setup Errors

- “npm not found” → install Node
- wrong folder → `cd minitasks`
- port busy → change port or close other app

Setup Entry Point

File: src/index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import "./styles/app.css";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

```
src/
  api/
  components/
  pages/
  store/
  styles/
  App.jsx
  index.js
```

Explanation

- ▶ BrowserRouter wraps App
- ▶ React mounts into #root
- ▶ routing won't work without BrowserRouter

Common mistakes

- ▶ Import path wrong ("./App" vs "./App.jsx")
- ▶ Forgot BrowserRouter → routes don't work
Fix: show error message and how it looks.

Checkpoint

- ▶ App loads without errors after restart

Setup Routing

File: src/App.jsx

ProtectedRoute:

- /login
- /register
- /tasks

```
src/  
  api/  
  components/  
  pages/  
  store/  
  styles/  
  App.jsx  
  index.js
```

Common mistakes

- Wrong path strings (spaces like `"/ tasks "`!)
- Forgot to export component

Fix: show correct code and “why whitespace breaks route matching”.

```
import { Routes, Route, Navigate } from "react-router-dom";  
import Login from "../pages/Login";  
import Register from "../pages/Register";  
import Tasks from "../pages/Tasks";  
import ProtectedRoute from "../components/ProtectedRoute";  
  
export default function App() {  
  return (  
    <Routes>  
      <Route path="/" element={<Navigate to="/tasks" />} />  
      <Route path="/login" element={<Login />} />  
      <Route path="/register" element={<Register />} />  
      <Route  
        path="/tasks"  
        element={  
          <ProtectedRoute>  
            <Tasks />  
          </ProtectedRoute>  
        }  
      />  
    </Routes>  
  );  
}
```

Now navigation works structurally.

Authentication Store.

File: src/store/authStore.js

```
const KEY = "token";
```

```
export const getToken = () => localStorage.getItem(KEY);
```

```
export const setToken = (token) => localStorage.setItem(KEY, token);
```

```
export const clearToken = () => localStorage.removeItem(KEY);
```

```
export const isAuthenticated = () => Boolean(getToken());
```

This isolates auth logic from UI.

```
src/  
  api/  
  components/  
  pages/  
  ● store/  
  styles/  
  App.jsx  
  index.js
```

Explanation

- ▶ localStorage persists across refresh
- ▶ token key
- ▶ isAuthenticated = Boolean(token)

Common mistakes

- ▶ KEY mismatch (“Token” vs “token”)
- ▶ localStorage blocked in private mode
Fix: open DevTools → Application →
LocalStorage to verify

Checkpoint

run `localStorage.setItem("token","123")`
and `isAuthenticated()` becomes true

ProtectedRoute

File: src/components/ProtectedRoute.jsx

```
import { Navigate } from "react-router-dom";
import { isAuthenticated } from "../store/authStore";

export default function ProtectedRoute({ children }) {
  if (!isAuthenticated()) return <Navigate to="/login" replace />;
  return children;
}
```

- ▶ Now routing is protected.
- ▶ We still have no API.

```
src/
  api/
  components/
  pages/
  store/
  styles/
  App.jsx
  index.js
```

Explanation

- ▶ Children composition
- ▶ Redirect logic

Common mistakes

- ▶ using Navigate without replace
- ▶ forgetting to import isAuthenticated path correctly
Fix: show typical error “Module not found...”

Checkpoint

- ▶ When token missing → /tasks redirects to /login

Add Navbar (Optional but realistic UI)

- ▶ show logged-in vs logged-out links

Common mistakes

- ▶ using `<a>` instead of `<Link>`
Fix: explain SPA reload

Checkpoint

- ▶ Navbar renders on all pages

API Layer Foundation

File: src/api/client.js

```
src/  
● api/  
  components/  
  pages/  
  store/  
  styles/  
  App.jsx  
  index.js
```

Now backend connection layer exists.
We still don't build UI logic yet.

```
import { getToken } from "../store/authStore";  
  
const API_BASE = process.env.REACT_APP_API_BASE ||  
"http://localhost:8000";  
  
export async function apiFetch(path, { method = "GET", body } = {}) {  
  const token = getToken();  
  
  const res = await fetch(API_BASE + path, {  
    method,  
    headers: {  
      "Content-Type": "application/json",  
      ...(token ? { Authorization: "Bearer " + token } : {}),  
    },  
    body: body ? JSON.stringify(body) : undefined,  
  });  
  
  const data = await res.json().catch(() => null);  
  
  if (!res.ok) {  
    throw new Error(data?.detail || data?.error || "Request failed");  
  }  
  
  return data;  
}
```

Explanation

- ▶ base URL
- ▶ headers
- ▶ token injection
- ▶ error handling

Common mistakes

- ▶ forgetting Content-Type
- ▶ sending body without `JSON.stringify`
- ▶ not checking `res.ok`
Fix: show correct pattern “Loading/Error/Success”.

Checkpoint

- ▶ When calling `apiFetch` to nonexistent server → shows readable error (not crash)

Auth API files api/auth.js

- ▶ loginApi
- ▶ registerApi

Common mistake

- ▶ wrong endpoint path
Fix: show “404 not found” in Network tab

```
src/  
● api/  
  components/  
  pages/  
  store/  
  styles/  
  App.jsx  
  index.js
```

Pages: Login & Register

Now we build UI forms.

File: src/pages/Login.jsx

Write:

- ▶ useState for email/password
- ▶ onSubmit
- ▶ call loginApi
- ▶ setToken
- ▶ redirect

Controlled inputs.
State drives UI.

```
src/  
  api/  
  components/  
  ● pages/  
    store/  
    styles/  
  App.jsx  
  index.js
```

Mistakes:

- ▶ forgetting `e.preventDefault()` → page reloads
- ▶ calling handler immediately
`onClick={fn()}`
Fix: `onClick={fn}`

Checkpoint

- ▶ form inputs work and state updates (console log)
- ▶ clicking submit does not reload page

Tasks Page State + useEffect (async pattern)

File: src/pages/Tasks.jsx

Start simple:

- ▶ `const [tasks, setTasks] = useState([]);`
`const [loading, setLoading] = useState(true);`
`const [error, setError] = useState("");`

Add:

- ▶ `useEffect(() => {`
 `load();`
`}, []);`

Component lifecycle.
Mount → fetch data.

```
src/  
  api/  
  components/  
  ● pages/  
    store/  
    styles/  
  App.jsx  
  index.js
```

useEffect runs on mount

When does it run?

```
useEffect(() => { ... }, [])
```

Runs **once after first render**

Equivalent to componentDidMount in class components

Used for: data fetching, subscriptions, initialization

Dependency Array

[] → run once (on mount)

[value] → run when value changes

No array → run on every render

Important Rule

The effect runs **after rendering**, not before.

React flow:

Render → Commit to DOM → Run Effects

Common mistakes

infinite loop in useEffect (state in deps updated inside effect)

Fix slide: show wrong + corrected code

Checkpoint

- ▶ tasks load (from mock)

Add CRUD Logic

Add the Following Operations:

Create Task → Add new item to state and backend

Toggle Task → Update task completion status

Delete Task → Remove task from state and backend

Optimistic UI Updates Pattern

Update UI immediately

Send API request

If request fails → rollback state

Why Use Optimistic Updates?

Improves perceived performance

Reduces UI latency

Provides smoother user experience

Requires rollback strategy for consistency

Add Filtering

```
const filtered = useMemo(() => {  
  if (filter === "active") return tasks.filter(t => !t.done);  
  if (filter === "done") return tasks.filter(t => t.done);  
  return tasks;  
}, [tasks, filter]);
```

Derived state concept.

Common mistake

- ▶ storing filtered list as separate state (causes inconsistencies)
Fix: derived state from source tasks

Checkpoint

- ▶ filter changes what is rendered

Debugging Toolkit

1 Console

Read error stack traces carefully

Use `console.log()` strategically

Identify undefined values and runtime errors

2 Network Tab

Inspect request URL and method

Check status codes (200, 401, 404, 500)

Verify request headers (Authorization)

Inspect response payload

3 Application / Storage

Check localStorage and session data

Verify token presence

Detect incorrect key names

4 React DevTools

Inspect component tree

Check props and state values

Track re-renders

Debugging Rule

Do not guess.

Inspect → Isolate → Verify → Fix.

Typical errors and how to read them

1 Module Not Found

Meaning:

Incorrect import path or missing file.

How to debug:

Check relative path (../ vs ./)

Verify file name and extension

Confirm file actually exists

2 401 Unauthorized

Meaning:

Request sent without valid authentication.

How to debug:

Check if token exists in localStorage

Verify Authorization: Bearer <token> header

Confirm token is not expired

Debugging Principle

Read the error message fully.

Identify the layer:

UI → Network → Backend → Configuration.

3 CORS Error

Meaning:

Backend does not allow requests from your frontend origin.

How to debug:

Check browser console message

Configure backend CORS settings

Ensure correct backend URL

Mock Mode

Why Mock Mode?

Frontend development should not depend on backend readiness

Enables parallel development

Allows testing UI logic independently

Reduces blocking between teams

Implementation

`const USE MOCK = true;`

`true` → Use local mock data

`false` → Use real API

Architectural Principle

Decouple UI logic from data source.

UI should not care whether data comes from:

Mock data

REST API

GraphQL

WebSocket

Production Transition

When backend is ready:

Set `USE MOCK = false`

Ensure API contract matches frontend expectations

Connecting the Real Backend (Django Integration)

Step 1 — Configure Environment

Create a `.env` file:

```
REACT_APP_API_BASE=http://localhost:8000
```

This defines where your frontend sends requests.

Step 2 — Disable Mock Mode

```
const USE MOCK = false;
```

Switch from local mock data to real API.

Step 3 — Ensure API Contract Matches

Backend must implement the expected endpoints:

- GET /api/tasks/
- POST /api/tasks/
- PATCH /api/tasks/:id/
- DELETE /api/tasks/:id/
- POST /api/auth/login/
- POST /api/auth/register/

Frontend and backend communicate through a contract.

If the contract changes, both sides must be updated.

CORS + JWT header

1. CORS (Cross-Origin Resource Sharing)

- Browser security mechanism
- Blocks requests between different origins (e.g., `localhost:3000` → `localhost:8000`)
- Must be explicitly allowed on the backend

In Django:

- Install `django-cors-headers`
 - Add frontend origin to allowed list
- Without CORS configuration → browser blocks the request.

Why Both Matter

- CORS allows the browser to send the request
 - JWT header authenticates the user
- One controls **access between origins**,
the other controls **access between users**.

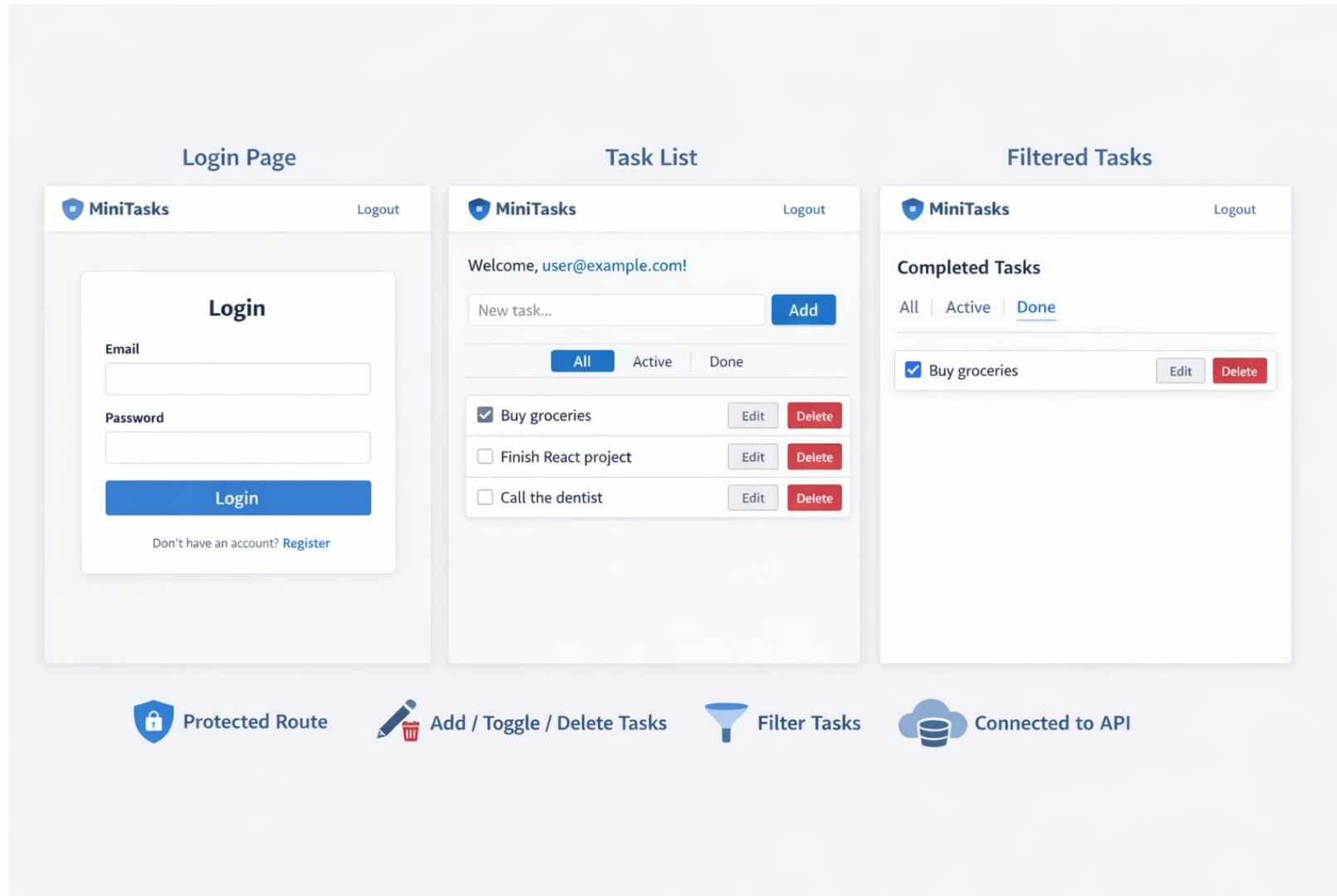
2. JWT Authorization Header

Frontend sends token in request header:

`Authorization: Bearer <token>`

- `Bearer` means access is granted to whoever holds the token
- Backend validates token signature
- If invalid → 401 Unauthorized

Results



PART 1 - QCM

The background of the slide features a series of overlapping triangles and lines in various shades of blue, ranging from light sky blue to a deep navy blue. These geometric shapes are primarily located on the right side of the slide, creating a modern, abstract design. The text 'PART 1 - QCM' is positioned on the left side of the slide, set against a plain white background.

30 TEST QUESTIONS

1. What is React primarily responsible for?

- A) Database management
- B) Server-side authentication
- C) User interface rendering
- D) Network security

2. What triggers a re-render in React?

- A) Changing local variables
- B) Changing state
- C) `Console.log()`
- D) Importing a file

3. Props in React are:

- A) Mutable
- B) Global
- C) Read-only
- D) Stored in database

4. What is the correct way to update state?

- A) state = newValue
- B) Direct assignment
- C) Using the setter function
- D) Modifying DOM manually

5. What is the purpose of the key prop in lists?

- A) Styling
- B) Security
- C) Performance and identity tracking
- D) Authentication

6. What happens if you use <a> instead of <Link> in React Router?

- A) Nothing changes
- B) State resets and full page reload occurs
- C) Faster navigation
- D) Backend error

7. What does useEffect primarily handle?

- A) Rendering UI
- B) Side effects
- C) Props validation
- D) Styling

8. What causes infinite loops in useEffect?

- A) Missing JSX
- B) Updating state that is also a dependency
- C) Using props
- D) Returning null

9. What is a controlled input?

- A) Input validated by backend
- B) Input managed by DOM only
- C) Input whose value is stored in React state
- D) Disabled input

10. Which HTTP method corresponds to "Create" in CRUD?

- A) GET
- B) POST
- C) DELETE
- D) PATCH

11. REST APIs are based on:

- A) XML-only communication
- B) Stateless client-server architecture
- C) Encrypted sessions
- D) WebSockets

12. What is JSON?

- A) Database system
- B) Programming language
- C) Data exchange format
- D) React library

13. Why is JSON preferred over XML in modern web apps?

- A) It supports namespaces
- B) It is more secure
- C) It is lighter and easier to parse
- D) It replaces REST

14. What does Fetch return?

- A) JSON directly
- B) HTML
- C) Promise
- D) Object

15. Why doesn't Fetch automatically throw HTTP errors?

- A) Because of REST rules
- B) Because response.ok must be checked manually
- C) Because of JSON
- D) Because of JSX

16. What advantage does Axios have over Fetch?

- A) Built-in routing
- B) Automatic JSON parsing and interceptors
- C) Faster rendering
- D) Database integration

17. What are the three recommended UI states when fetching data?

- A) Idle, Running, Done
- B) Loading, Error, Success
- C) True, False, Null
- D) Fetch, Parse, Render

18. What is JWT?

- A) Encryption protocol
- B) JSON database
- C) Signed authentication token
- D) React hook

19. JWT is:

- A) Encrypted by default
- B) Signed but not encrypted
- C) Only for backend
- D) Stored in database

20. How is JWT usually sent to backend?

- A) In URL
- B) In body
- C) In Authorization header
- D) In cookies automatically

21. What does “Bearer” mean in Authorization header?

- A) Encrypted
- B) Whoever holds the token can access
- C) Admin-only
- D) Expired

22. What is a Single Page Application?

- A) One HTML page dynamically updated
- B) Static website
- C) Backend-only system
- D) XML-based app

23. What is prop drilling?

- A) Passing props deeply through many levels
- B) Updating state incorrectly
- C) Using hooks improperly
- D) Routing error

24. What is the role of BrowserRouter?

- A) Handles API requests
- B) Enables URL-based routing
- C) Stores token
- D) Controls state

25. Which is true about state?

- A) It is global automatically
- B) It cannot change
- C) It is internal to component
- D) It replaces props

26. What is REST focused on?

- A) Sessions
- B) Resources and HTTP methods
- C) Encryption
- D) SOAP standards

27. GraphQL allows:

- A) Only POST requests
- B) Strict XML schemas
- C) Requesting exactly required data
- D) No authentication

28. Why should sensitive data not be stored in JWT?

- A) Because JWT expires
- B) Because JWT can be decoded
- C) Because JSON is unsafe
- D) Because React blocks it

29. What is the correct UI behavior when API fails?

- A) Reload page
- B) Ignore error
- C) Show error state
- D) Remove component

30. What is the main architectural benefit of separating API layer?

- A) Faster UI
- B) Better styling
- C) Centralized communication logic
- D) Less JSX

PART 2 — ANALYTICAL QUESTIONS

1. A component fetches data inside `useEffect` but creates an infinite loop. Explain exactly why this happens and how to fix it.
2. Compare REST and GraphQL for a large e-commerce application. Which would you choose and why?
3. Explain why React uses declarative rendering instead of imperative DOM manipulation. What architectural benefits does this provide?
4. Explain why separating API logic from UI components improves scalability in large applications.

ANSWER KEY - PART 1

- | | |
|-------|-------|
| 1. C | 16. B |
| 2. B | 17. B |
| 3. C | 18. C |
| 4. C | 19. B |
| 5. C | 20. C |
| 6. B | 21. B |
| 7. B | 22. A |
| 8. B | 23. A |
| 9. C | 24. B |
| 10. B | 25. C |
| 11. B | 26. B |
| 12. C | 27. C |
| 13. C | 28. B |
| 14. C | 29. C |
| 15. B | 30. C |

ANSWER KEY - PART 2

1. **A component fetches data inside `useEffect` but creates an infinite loop. Explain exactly why this happens and how to fix it.**

An infinite loop in `useEffect` happens when the effect updates a state variable that is also included in its dependency array.

In React, whenever state changes, the component re-renders. After re-rendering, React checks the dependency array of `useEffect`. If one of the dependencies has changed, the effect runs again. If the effect itself updates that same dependency, the cycle continues indefinitely.

This creates repeated renders and continuous execution of the effect. The most common cause is placing a state variable inside the dependency array while also modifying it inside the effect. To fix this, the dependency array must reflect only values that truly control the effect.

If the effect should run only once, an empty dependency array `[]` should be used.

Another solution is restructuring the logic so that the effect does not directly modify its own dependency. Proper understanding of state lifecycle and dependency management prevents infinite loops.

REST vs GraphQL in Large E-commerce Applications

REST and GraphQL are two different approaches to API design. REST is based on resources and HTTP methods, with multiple endpoints such as /products, /orders, and /users.

GraphQL uses a single endpoint and allows the client to request exactly the data it needs. In REST, responses are predefined by the server, which can lead to over-fetching or under-fetching.

GraphQL solves this by allowing flexible queries.

However, REST is simpler to implement and easier to cache using standard HTTP caching mechanisms. For large e-commerce applications with heavy traffic, caching is extremely important for performance.

REST integrates well with CDNs and scaling infrastructure.

GraphQL may be better when frontend views require highly dynamic and nested data. In most large-scale production systems, REST is often preferred due to its stability, simplicity, and scalability.

Declarative Rendering in React

React uses declarative rendering because it improves predictability and maintainability. In declarative programming, developers describe what the UI should look like based on state.

React then determines how to update the DOM efficiently. This contrasts with imperative programming, where developers manually manipulate DOM elements step by step.

Manual DOM manipulation increases complexity and risk of inconsistent UI states. Declarative rendering ensures that the UI is always synchronized with the application state. When state changes, React automatically re-renders the component.

This reduces bugs caused by manual updates. It also improves code readability and separation of concerns. Architecturally, declarative rendering leads to more scalable and maintainable frontend systems.

Separating API Logic from UI Components

Separating API logic from UI components improves scalability and maintainability. When API calls are placed directly inside components, the code becomes tightly coupled and harder to manage.

This leads to duplication of fetch logic across multiple components. Centralizing API communication in a separate layer creates a single source of truth. It also allows consistent error handling and token management.

UI components then focus only on rendering and state management.

This separation follows the principle of separation of concerns. In large applications, multiple developers may work on different layers independently. It also simplifies testing, since API logic can be tested separately from UI rendering.

Architecturally, this pattern results in cleaner, more scalable systems that are easier to extend.